

SimOffice

Development Architecture & Technical Specification

Version 0.3 — May 2026

Christopher Gordon Phillips
LumenHelix Solutions
chris@oiq.to · lumenhelix.com

Abstract

SimOffice is an isometric agent workspace simulator that borrows interaction paradigms from classic simulation and strategy games—particularly *The Sims*, early *Grand Theft Auto*, and *StarCraft*—to create an intuitive, visual interface for spawning, configuring, and observing autonomous AI agents in a virtual office environment. Agents are powered by multi-model LLM backends (Claude, GPT-4o, Llama, Mistral) with dynamic task-based routing managed by a senior agent. The system features a drag-and-drop component library where placing objects grants agents corresponding capabilities, prefabricated environment templates that are fully deconstructable and remixable, and a community sharing layer for collaborative world-building. This document defines the complete development architecture, from rendering pipeline through agent cognition to token-efficiency integration via the OmniCube compression framework.

Contents

1	Vision & Motivation	2
1.1	Design Influences	2
2	System Architecture Overview	2
2.1	Data Flow	3
3	Rendering Engine	3
3.1	Isometric Projection	3
3.2	Rendering Pipeline	4
3.3	Visual Design Language	4
4	Environment System	5
4.1	The Office Grid	5
4.2	Sandbox Mode	5
4.3	Live Mode	5
4.4	Onboarding Flow	6
5	Component Library: Furniture as Capability	6

6	Agent Architecture	6
6.1	Prefab Agent Roster	6
6.2	Agent State Model	7
6.3	Task Assignment Flow	7
6.4	Contextual Activation & Onboarding Guidance	8
7	Multi-Model Routing Architecture	8
7.1	Supported Providers	8
7.2	Routing Strategy: Hybrid Task + Override	8
7.3	CFO as Cost Oracle	9
7.4	Demo Mode	9
8	Token Efficiency & OmniCube Integration	9
9	Prefab Environment System	9
9.1	Template Architecture	9
9.2	Planned Templates	10
9.3	Deconstructability & Remixing	10
10	Community & Sharing Layer	10
11	Technology Stack	11
12	Development Roadmap	11
13	Prior Art & Related Work	11
14	Conclusion	12

1 Vision & Motivation

The current landscape of AI agent orchestration tools falls into two categories: enterprise backends that automate tasks invisibly (Moveworks, Ema, Salesforce Agentforce), and academic simulations that lack visual charm or accessibility (Mesa, AnyLogic). Neither makes agent behavior *watchable*, *shareable*, or *enjoyable*.

SimOffice occupies the gap between these worlds. It treats agent orchestration as a *spatial*, *visual*, *social experience*—the same paradigm that made The Sims one of the best-selling game franchises in history.

Core Thesis

Game interfaces evolved over decades to make complex systems legible and interactive. AI agent orchestration is a complex system. Therefore, the most intuitive agent interface is a game-inspired one.

1.1 Design Influences

Three game design lineages converge in SimOffice:

1. **The Sims** — Character creation as agent parameterization. The “build mode / live mode” duality. Object placement as capability granting. The emotional legibility of watching characters move through space with purpose.
2. **Grand Theft Auto 1/2** — The isometric (45-degree) camera angle that provides spatial depth without 3D rendering overhead. Readable character movement in a 2D-projected space. The sense of a living, self-directed world.
3. **StarCraft / RTS Games** — Unit spawning as agent instantiation. Building construction as capability deployment. Resource management as token-cost optimization. The “fog of war” parallel to agent information boundaries.

The AgentCraft project (2025–2026) demonstrated that RTS interfaces work for agent orchestration. Stanford’s Generative Agents paper (Park et al., 2023) proved that LLM-powered agents can produce believable autonomous behavior in a Sims-like sandbox with memory streams and reflection architectures. SimOffice synthesizes both insights with a focus on *workplace productivity*, *community sharing*, and *multi-model cost optimization*.

2 System Architecture Overview

SimOffice comprises five architectural layers:

Layer Stack

1. **Rendering Layer** — Isometric Canvas2D engine with gradient shading, drop shadows, glow effects, and ambient animation.
2. **Environment Layer** — Spatial model (grid, rooms, furniture, capability objects), physics-free navigation, and the component library.
3. **Agent Layer** — Agent definitions, role archetypes, task assignment, movement state, and personality/behavior parameters.
4. **Cognition Layer** — Multi-model LLM routing, system prompts, memory streams, inter-agent communication, and the senior-agent task router.
5. **Efficiency Layer** — Token cost tracking (CFO agent), local model fallback, OmniCube compression integration, and community-shared optimization profiles.

2.1 Data Flow

The runtime data flow follows a perception–decision–action cycle:

1. **Environment tick** advances the simulation clock. The speed multiplier ($1\times$, $2\times$, $4\times$) scales the tick rate.
2. **Task timer** fires for each active agent. When an agent’s timer expires, the **Senior Agent** (CEO) selects a task from the agent’s task pool, considering CFO cost advisories.
3. **Movement** triggers: the agent walks to the spatial location associated with the task (desk, meeting room, server rack, filing cabinet).
4. **LLM call** fires asynchronously. The agent’s status transitions to “thinking.” The routing layer selects the appropriate model based on task type, agent assignment, and cost constraints.
5. **Response** arrives and is displayed as a speech bubble, logged to the task feed, and stored in the agent’s memory stream (future version).
6. **State update** propagates: agent panel refreshes, CFO tracks token cost delta, rendering layer updates agent position and status indicator.

3 Rendering Engine

3.1 Isometric Projection

SimOffice uses a standard 2:1 isometric projection with tile dimensions $T_w = 64$ px, $T_h = 32$ px. The forward transform from grid coordinates (g_x, g_y) to screen coordinates (s_x, s_y) at height h above the ground plane is:

$$s_x = O_x + (g_x - g_y) \cdot \frac{T_w}{2} \tag{1}$$

$$s_y = O_y + (g_x + g_y) \cdot \frac{T_h}{2} - h \tag{2}$$

where (O_x, O_y) is the screen-space origin (typically the center-top of the viewport). The inverse transform for hit-testing (click $\rightarrow$ grid cell) is:

$$g_x = \left\lfloor \frac{\Delta x}{T_w} + \frac{\Delta y}{T_h} \right\rfloor \cdot \frac{1}{2} \quad (3)$$

$$g_y = \left\lfloor \frac{\Delta y}{T_h} - \frac{\Delta x}{T_w} \right\rfloor \cdot \frac{1}{2} \quad (4)$$

where $\Delta x = s_x - O_x$, $\Delta y = s_y - O_y$.

3.2 Rendering Pipeline

The v0.3 renderer uses raw HTML5 Canvas 2D (no game engine dependency) with HiDPI scaling via `devicePixelRatio`. The draw order follows the painter's algorithm:

1. Clear canvas, fill background.
2. Draw floor tiles (back-to-front by $g_x + g_y$), with carpet zone differentiation and window-light overlay.
3. Draw walls: left wall (y-facing panels with luminance falloff), back wall (x-facing panels with window sky gradients and mullion details), baseboard lines, and wall-top edge highlights.
4. Collect all furniture items and active agents into a single depth-sorted array ($g_x + g_y$ ascending).
5. For each object: draw drop shadow (dark ellipse at base), then draw the object. Furniture uses gradient-shaded box faces with edge highlights. Desks additionally render monitors with animated screen glow and keyboard silhouettes. Server racks render blinking LED arrays. Plants render leaf clusters.
6. Agents render with gradient-shaded torsos, skin-tone heads, hair detail, breathing idle animation (sin bob), and glowing status indicator dots.

3.3 Visual Design Language

Element	Treatment	Technique
Floor tiles	Alternating warm wood tones, carpet zone in blue-gray	Flat fill + grid stroke
Furniture faces	Linear gradient (light at top, dark at bottom)	<code>createLinearGradient</code>
Drop shadows	Soft dark ellipses under all objects	<code>globalAlpha</code> + <code>ellipse</code>
Monitor screens	Animated cyan glow with sin-based flicker	<code>shadowBlur</code> + composite
Server LEDs	Phase-offset sin blink pattern (green + red)	Per-frame redraw
Window light	Low-opacity warm overlay on floor tiles near window	<code>globalAlpha</code> overlay
Agent bodies	Gradient-shaded jacket, rounded proportions	<code>roundRect</code> + gradient
Agent status	Glowing colored dot with <code>shadowBlur</code> halo	Canvas shadow API

Table 1: Rendering techniques by visual element.

4 Environment System

4.1 The Office Grid

The base environment is a 10×8 isometric tile grid. Walls enclose the left ($x=0$) and back ($y=0$) edges. Windows span tiles $x \in [3, 6]$ on the back wall, providing a sky gradient with mullion details and casting a warm light overlay on nearby floor tiles.

The grid supports:

- **Floor zones:** carpet (meeting area), wood (work area), with distinct tile colors per zone.
- **Furniture placement:** any grid cell can receive a furniture item of arbitrary tile footprint ($w \times d$).
- **Agent stations:** each agent has a designated desk position (grid coordinate pair). Agents navigate to task-specific destinations and return to their station when idle.

4.2 Sandbox Mode

Sandbox mode presents the office grid with two side panels:

- **Left panel (Furnish):** A component palette listing all available furniture items. Clicking an item selects it; clicking a floor tile places it. A hover-highlight (red diamond outline) previews the placement cell.
- **Right panel (Office):** Displays office metadata (name, type, item count), agent activation toggles, and the “GO LIVE” transition button.

All prefab furniture is visible in Sandbox mode and can be rearranged or supplemented. The Sandbox is the deconstructable workspace described in the design philosophy—every prefab can be pulled apart, modified, and reassembled.

4.3 Live Mode

Live mode activates agents and starts the simulation clock. The interface presents:

- **Top bar:** Office name, simulation date/time, API status badge, settings and sandbox return buttons.
- **Left panel (Agents):** Active agent cards showing name, role, current task, status indicator (working / meeting / moving / thinking), and assigned model tag.
- **Right panel (Task Feed):** Scrollable chronological log of agent LLM outputs—who said it, what task triggered it, timestamp, and the full response text.
- **Bottom bar:** Time controls (Pause / $1\times$ / $2\times$ / $4\times$), simulation clock, and latest task assignment ticker.

4.4 Onboarding Flow

The onboarding is a conversational chatbot hybrid that combines natural-language prompts with quick-select option buttons:

1. “What kind of office are you building?” — Guided templates (Law Firm, Tech Startup, Design Studio, etc.) *or* open-ended “Tell me what I need.”
2. “What do you want to call it?” — Free-text input.
3. “How big is your team?” — Size bracket selection.
4. “What’s your primary focus?” — Product dev, client work, research, operations.

After onboarding, the system sets the office name/type and transitions to Sandbox mode. Future versions will use onboarding answers to pre-activate contextually relevant agents and suggest appropriate environment templates.

5 Component Library: Furniture as Capability

A central design principle of SimOffice is that **placing an object in an agent’s workspace grants that agent access to the object’s capability**. This decouples capability management from agent configuration—you don’t set permissions in a menu; you drag a phone onto a desk.

Object	Palette ID	Capability Unlocked
Desk	<code>desk</code>	Workspace anchor; required for agent station assignment
Computer/Monitor	<code>monitor</code>	Network access, LLM API calls, web browsing
Phone	<code>phone</code>	Voice call simulation, external communication
Mailbox	<code>mail</code>	Email send/receive, notification dispatch
Filing Cabinet	<code>cab</code>	Document storage, retrieval, archival
Server Rack	<code>srv</code>	Data processing, compute jobs, pipeline execution
Meeting Table	<code>tbl</code>	Multi-agent synchronous collaboration
Whiteboard	<code>board</code>	Shared ideation, planning artifact creation
Printer	<code>prn</code>	Document output, report generation
Plant	<code>plnt</code>	Aesthetic only (morale modifier in future versions)

Table 2: Component library: objects and their granted capabilities.

Design Principle: Object Semantics

The component library is *not* decorative. Each object is a first-class capability token. When an agent needs to send an email, the system checks whether a mailbox object exists within the agent’s accessible workspace. If not, the capability is unavailable. This creates a legible, spatial model of agent permissions.

6 Agent Architecture

6.1 Prefab Agent Roster

The beta ships with a C-suite starter cast plus support roles. All agents start *dormant* and activate contextually during onboarding or manually in Sandbox mode.

Role	Name	Model	Primary Domain	Status
CEO	Alex Chen	Claude Sonnet	Strategy, cross-team alignment, decision-making	Active (v0.3)
CTO	Marcus Webb	Claude Opus	Architecture, code review, technical decisions	Active (v0.3)
CFO	Priya Singh	GPT-4o-mini	Token cost tracking, budget analysis, model economics	Active (v0.3)
CIO	James Torres	Llama (local)	Infrastructure, data pipelines, system monitoring	Active (v0.3)
Admin	Sam Riley	Mistral (local)	Scheduling, filing, coordination, logistics	Active (v0.3)
HR	—	TBD	Recruitment, onboarding, policy compliance	Planned
Trainer	—	TBD	Agent skill development, prompt optimization	Planned
Analyzer	—	TBD	Data analysis, report generation, pattern detection	Planned

Table 3: Agent roster with model assignments and development status.

6.2 Agent State Model

Each agent maintains the following runtime state:

- **Position** (s_x, s_y): Current screen coordinates (updated per frame during movement).
- **Target** (t_x, t_y): Destination screen coordinates for current movement.
- **Status**: One of **work** (at desk), **meet** (at meeting table), **move** (in transit), **think** (awaiting LLM response).
- **Task**: Current task label (e.g., “Architecture review,” “Token cost analysis”).
- **Task timer**: Countdown until next task assignment (randomized interval, 350–800 ticks).
- **LLM pending**: Boolean flag preventing concurrent API calls per agent.
- **Last output**: Most recent LLM response text.
- **Model assignment**: Which LLM provider/model handles this agent’s cognition.

6.3 Task Assignment Flow

1. Agent’s task timer expires (and no LLM call is in flight).
2. A task is selected from the agent’s task pool (uniform random in v0.3; senior-agent-routed in future).
3. Movement target is set to the task’s spatial destination. Status transitions to the task’s type (work/meet/move).
4. An asynchronous LLM call fires:
 - (a) Status temporarily transitions to **think**; speech bubble shows “...”

- (b) The LLM receives a role-specific system prompt and a task-contextualized user message.
 - (c) On response: speech bubble displays the text; task feed logs it; status reverts.
5. A 4-second cooldown prevents successive LLM calls for the same agent.

6.4 Contextual Activation & Onboarding Guidance

Agents are *dormant by default*. As users build their office environment in Sandbox mode, the system suggests activating relevant agents:

Example: Adding a server rack triggers: “Your office now has compute infrastructure. Consider activating James Torres (CIO) to manage it.”

This progressive disclosure prevents information overload while teaching users the system’s full capabilities through doing.

7 Multi-Model Routing Architecture

7.1 Supported Providers

SimOffice is model-agnostic by design. The v0.3 beta supports:

Provider	Models	Access	Notes
Anthropic	Claude Sonnet 4.6, Opus 4.6	API key	Primary models for strategic/technical agents
OpenAI	GPT-4o-mini	API key	Cost-efficient for analytical tasks
Meta	Llama 3.2	Ollama (local)	Zero-cost inference for infrastructure tasks
Mistral	Mistral	Ollama (local)	Zero-cost inference for administrative tasks
xAI	Grok	API key (planned)	Planned for future integration
Google	Gemini	API key (planned)	Planned for future integration

Table 4: Supported LLM providers and access methods.

7.2 Routing Strategy: Hybrid Task + Override

The routing architecture combines three strategies:

- Task-based routing** (default): Each task type has a preferred model class. Administrative tasks route to local models; strategic decisions route to Claude; financial analysis routes to GPT-4o.
- Per-agent override**: Users can manually pin any agent to any model, overriding the default routing.
- Senior-agent dynamic routing** (essential, future): The CEO agent observes the CFO’s cost reports and dynamically adjusts routing decisions in real time. This is not a menu toggle—it is an *emergent behavior* of the senior agent’s cognition.

7.3 CFO as Cost Oracle

The CFO agent (Priya Singh) serves a dual role:

- **Cost tracking:** Monitors token consumption per agent, per model, per time window. Generates cost projections and budget alerts.
- **Advisory output:** Produces natural-language cost recommendations that the CEO agent consumes as decision inputs. Example: “Claude Sonnet costs $3\times$ more per token than Haiku. Recommend downgrading Admin to Haiku for scheduling tasks.”

7.4 Demo Mode

When no API keys are configured, agents cycle through curated pre-written responses that simulate realistic LLM output. This ensures the system is demonstrable without any external dependencies. A badge in the Live mode top bar indicates “DEMO” or “API” depending on key status.

8 Token Efficiency & OmniCube Integration

Token cost is the primary operational expense of a multi-agent system running continuous LLM calls. SimOffice addresses this through four mechanisms:

1. **Local model fallback:** Route low-complexity tasks to locally-running Llama or Mistral via Ollama, incurring zero API cost.
2. **Response length constraints:** System prompts enforce strict word limits (30–50 words), keeping token consumption per call minimal.
3. **Call cooldown:** A per-agent minimum interval (4 seconds in v0.3) prevents token waste from rapid-fire calls at high simulation speeds.
4. **OmniCube compression** (planned): Integration with LumenHelix’s reversible computing compression technology for prompt and context-window compression, reducing token counts at the encoding layer.

OmniCube Integration Point

The OmniCube framework, built on the CORE-32 reversible computation architecture, provides a natural compression layer for agent context windows. By encoding agent memory streams and system prompts through the $\delta_{32}(x) = x \oplus 16$ involution-based compression pipeline, context can be transmitted to LLM providers with reduced token counts while preserving semantic fidelity. This integration is planned for v0.5.

9 Prefab Environment System

9.1 Template Architecture

A prefab environment is a JSON-serializable bundle containing:

- **Room definition:** Grid dimensions, wall configuration, floor zone map.

- **Furniture layout:** Array of placed objects with grid positions and configurations.
- **Agent preset:** Which agents are active, their desk assignments, model overrides.
- **Task pool overrides:** Domain-specific task definitions (e.g., a law firm’s CEO reviews “case strategy” instead of “product roadmap”).
- **Metadata:** Template name, description, author, version, category tags.

9.2 Planned Templates

- Generic Office (v0.3, shipped)
- Law Firm
- Tech Startup
- Design Studio
- Marketing Agency
- Hedge Fund / Trading Floor
- Home Office (solo)
- Data Center / NOC
- Research Lab
- Customer Service Center

9.3 Deconstructability & Remixing

Every prefab environment is fully deconstructable in Sandbox mode. Users can:

- Inspect how any agent is configured.
- Rearrange, add, or remove furniture.
- Modify agent roles, model assignments, or task pools.
- Export the modified environment as a new template.
- Share templates with the community.

This creates an **open-source world-building economy** where expertise is encoded in shareable, forklable environment configs. A user can import “Corporate Law Firm v2 with paralegal focus” and adapt it for their own use case.

10 Community & Sharing Layer

The community layer (planned for v0.4+) enables:

1. **Template marketplace:** Browse, search, and install community-created environment templates.
2. **Component sharing:** Individual furniture items, agent configurations, and task pools can be shared independently of full environments.
3. **Scenario replays:** Export a completed simulation run as a shareable replay. Others can watch how your configured agents handled a scenario, then fork and modify the setup.
4. **Versioning:** Templates support semantic versioning. Forked templates maintain provenance links to their parent.

11 Technology Stack

Layer	Technology	Status
Rendering	Raw HTML5 Canvas 2D, HiDPI-aware, no framework dependency	Shipped (v0.3)
UI Framework	Vanilla HTML/CSS/JS with Google Fonts (VT323 + Outfit)	Shipped (v0.3)
Agent Backend	Multi-provider LLM client (fetch-based, browser-native)	Shipped (v0.3)
Local Inference	Ollama (llama3.2, mistral) via localhost:11434	Shipped (v0.3)
Persistence	localStorage for API keys; JSON export/import for office configs	Partial (v0.3)
Compression	OmniCube / CORE-32 token compression layer	Planned (v0.5)
Community	GitHub-based template sharing; future: hosted marketplace	Planned (v0.4)
Mobile	PWA wrapper with responsive canvas scaling	Planned (v0.6)

Table 5: Technology stack by architectural layer.

12 Development Roadmap

Version	Milestone	Deliverables
v0.1	Environment Foundation	Isometric room, furniture palette, drag-and-drop, sandbox/live modes
v0.2	LLM Integration	Real API calls per agent, demo mode, speech bubbles, task feed, settings
v0.3	Visual Polish	Canvas2D rewrite, gradient shading, shadows, glow effects, ambient animation
v0.4	Community Sharing	JSON export/import, GitHub template repo, template browser
v0.5	OmniCube Compression	Token compression integration, CFO real-time cost dashboard
v0.6	Senior Agent Router	CEO dynamic routing logic, CFO advisory consumption, emergent cost optimization
v0.7	Memory Streams	Per-agent long-term memory, reflection architecture, inter-agent awareness
v0.8	Multiple Environments	Multi-room offices, building-level navigation, department isolation
v0.9	Additional Prefabs	Law Firm, Tech Startup, Design Studio templates with domain-specific agents
v1.0	Production Release	Hosted platform, user accounts, template marketplace, mobile PWA

Table 6: Development roadmap from prototype to production.

13 Prior Art & Related Work

- **Generative Agents** (Park et al., Stanford/Google, 2023): 25 LLM-powered agents in a pixel-art sandbox with memory streams and reflection. Foundational reference architecture for SimOffice’s agent cognition layer.
- **AgentCraft** (2025–2026): RTS-style interface for AI agent orchestration. Demonstrated that game interfaces reduce cognitive load for agent management. SimOffice differs in its workplace/social focus versus AgentCraft’s command-and-control orientation.

- **Mesa** (Python): Academic agent-based modeling framework. Strong simulation capabilities but no visual interface or LLM integration.
- **AnyLogic**: Commercial multi-method simulation platform. Enterprise-grade but lacks the game-like accessibility and community sharing model.
- **SIMA / Genie 2** (Google DeepMind): Generalist AI agents for 3D virtual worlds and procedural world generation. Relevant for future procedural environment generation but operates at a different abstraction level.
- **FreeSims / OpenTS2**: Open-source Sims engine recreations. Potential asset pipeline or interaction-model references, though they require original game files.
- **Carnage3D**: Open-source GTA 1 reimplement. Reference for isometric rendering and 2D character movement in a city-scale environment.

14 Conclusion

SimOffice occupies a unique position in the AI agent tooling landscape: it is neither a dry enterprise platform nor an academic simulation framework, but a *game-inspired workspace* that makes agent behavior visible, interactive, and shareable. By borrowing proven interaction paradigms from The Sims (character creation, build/live duality), GTA (isometric spatial awareness), and StarCraft (unit spawning, building-as-capability), SimOffice transforms agent orchestration from a backend engineering problem into a visual, intuitive, and collaborative experience.

The v0.3 prototype demonstrates the core loop: onboard, build, activate, watch, iterate. The path from here to production runs through community sharing, OmniCube token compression, senior-agent emergent routing, and a growing library of domain-specific environment templates—each deconstructable, each remixable, each a building block in an open ecosystem of AI-powered workspaces.

SimOffice is a LumenHelix Solutions project.
Christopher Gordon Phillips · chris@oiq.to · lumenhelix.com